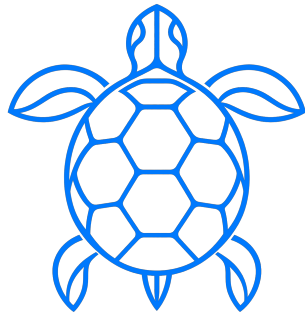




Testudo

User Manual

A native local-first macOS environment for structured professional and research work

Manual version: 1.2

Application version documented: 0.1.5 (build 6)

Platform: macOS, Apple Silicon

Date: October 2026

About this manual

This manual documents the current implemented form of **Testudo** , a native macOS application for organising complex professional, technical, academic and research work.

This edition documents Testudo 0.1.5 (build 6).

Manual version 1.2 reflects the current macOS 26 application baseline, the modern SwiftUI interface, reliable global New Entry routing, native sidebar behaviour, Liquid Glass Back and Forward detail navigation, the unified structural detail inspectors, and the current warning-free SwiftPM build workflow.

The manual focuses on the user-facing concepts and workflows of the application. It also explains the local-first storage model, authentication architecture, calendar integration, portability and installation process.

Testudo is under active development. Screens, labels and individual workflows may evolve between versions. Where the data model provides a stable conceptual distinction, this manual explains that distinction explicitly.

Terminology

In the current implementation, a chronological work-log entry inside the Work hierarchy is called an **Activity** . A scheduled **Calendar Event** is a separate object. The two concepts should not be confused.

Image used in this manual

The visual mark used on the cover is `Sources/Testudo/Resources/testudo_icon_blue.png` . This is also the default visual used for newly created Work Environments.

Contents

1	Introduction	1
1.1	What Testudo is	1
1.2	Local-first design	1
1.3	The principal data domains	2
2	Installation	3
2.1	System requirements	3
2.2	Quick local installation	3
2.3	Inspecting the installer first	4
2.4	Manual local build	4
2.5	Installing Apple developer tools	4
2.6	Updating Testudo	4
2.7	Why Testudo currently uses local compilation	4
3	First Launch and the Local User	6
3.1	The local Testudo profile	6
3.2	Local user versus Environment member	6
3.3	Portable user profile	7
4	Work Environments	8
4.1	Concept	8
4.2	The <code>.testudoenv</code> package	8
4.3	Environment manifest	8
4.4	Environment storage location	8
4.5	Creating or registering an Environment	9
4.6	Environment icon	9
4.7	Backing up an Environment	9
5	Authentication and Roles	10
5.1	Two authentication layers	10
5.2	Environment roles	10
5.3	Environment passwords	10
5.4	Security questions and password recovery	10
5.5	Stay signed in	10
5.6	Log Out and Sign Out	11
6	The Testudo Interface	12
6.1	Three-column navigation	12
6.2	Main navigation	12
6.3	Sidebar visibility	12
6.4	Detail navigation	12
6.5	Hierarchical presentation	13
6.6	Middle-column selection	13
7	Work	14
7.1	New Entry	14
7.2	Work item types	14
7.3	Tasks	14
7.4	Task states	14
7.5	Task dates	15
7.6	Sub-tasks	15
7.7	Notes	15

7.8 Activities	16
8 Relationships	17
8.1 Why relationships matter	17
8.2 Work relationship roles	17
8.3 Relationship inheritance	17
8.4 Related Work	17
9 Themes	19
9.1 Purpose	19
9.2 Theme hierarchy	19
9.3 Theme information	19
9.4 Themes and Work	20
9.5 Themes and entities	20
10 People	21
10.1 People directory	21
10.2 Names and ordering	21
10.3 Affiliations	21
10.4 Primary affiliation	21
10.5 Related Work	21
10.6 Person versus local profile	22
11 Groups and Organizations	23
11.1 Organizations	23
11.2 Organization metadata	23
11.3 Groups	24
11.4 Membership	24
11.5 Related Work	24
12 Calendar	25
12.1 Calendar Events	25
12.2 Calendar Event status	25
12.3 Testudo calendars	25
12.4 Apple Calendar and iCloud	25
12.5 iCal subscriptions	26
12.6 Secret iCal addresses	26
12.7 External calendars and Environment administration	26
12.8 Linking Calendar Events to Work	26
13 Time Zones and Date Handling	27
13.1 Device time zone	27
13.2 IANA time zones	27
13.3 Absolute timestamps	27
13.4 Displaying source and local time	27
13.5 Changing the selected source zone	27
14 History and Audit Information	28
14.1 Work history	28
14.2 Actor information	28
14.3 History time zones	28
15 Demo Environment	29
15.1 Purpose	29
15.2 Demo subject matter	29
15.3 Generated objects	29
15.4 Demo dates	30
15.5 Demo naming	30
16 Data Portability and Storage	31

16.1 What is portable	31
16.2 What is local to one installation	31
16.3 Moving a Work Environment	31
16.4 Using cloud-synchronised storage	31
17 Administration	32
17.1 Environment Administrator	32
17.2 Membership administration	32
17.3 Password administration	32
17.4 Environment appearance	32
17.5 Connected calendars	32
18 Security and Privacy	33
18.1 Local-first privacy model	33
18.2 Passwords	33
18.3 Sensitive external calendar information	33
18.4 Calendar permission	33
18.5 Portable files	33
19 Troubleshooting	34
19.1 The build fails with an old toolchain	34
19.2 SwiftPM build progress	34
19.3 Apple developer tools are missing	34
19.4 The automatic installer refuses to update	34
19.5 The Work Environment cannot be found	35
19.6 Apple Calendar does not appear	35
19.7 iCal synchronisation fails	35
19.8 Several old Testudo installer volumes appear	35
20 Recommended Working Practices	36
20.1 Use Tasks for actionable work	36
20.2 Use Notes for context	36
20.3 Use Activities for the work log	36
20.4 Use Calendar Events for scheduled time	36
20.5 Use Themes for semantic classification	36
20.6 Model People and Organizations explicitly	36
20.7 Preserve history	37
21 Technical Reference	38
21.1 Important file extensions	38
21.2 Source tree	38
21.3 Build scripts	38
21.4 Version information	38
21.5 Application build	39
21.6 Local DMG build	39
22 Glossary	40
A Command Reference	41
A.1 Install Testudo	41
A.2 Clone manually	41
A.3 Update source	41
A.4 Build local installer	41
A.5 Build application only	41
A.6 Check Xcode and Swift	41
A.7 Request Apple Command Line Tools	41
B Manual Source and Compilation	42
C Early Release Notice	43

Introduction

1.1 What Testudo is

Testudo is a native macOS application designed to bring several normally separate aspects of professional work into one structured environment.

Its core model combines:

- hierarchical Tasks and sub-tasks;
- Notes and chronological Activities;
- scheduled Calendar Events;
- People;
- Groups;
- Organizations;
- thematic classification through Themes;
- relationships between work and real-world entities;
- chronological history and audit information;
- portable, independent Work Environments.

The application is particularly suitable for situations in which work cannot be represented adequately by a flat task list.

Examples include:

- scientific research;
- research software engineering;
- laboratory and technical operations;
- academic collaborations;
- multi-institution projects;
- infrastructure administration;
- project coordination;
- seminar or working-group organisation;
- long-running professional activities with extensive history.

1.2 Local-first design

Testudo is designed as a local-first application.

There is no mandatory Testudo cloud service. The primary data of a Work Environment is stored in a portable `.testudoenv` package chosen by the user.

The package may be placed:

- on local storage;
- in iCloud Drive;
- in Dropbox;

- in OneDrive;
- or in another filesystem location available to macOS.

Testudo works with the selected package instead of importing its contents into a proprietary remote service.

A cloud-synchronised filesystem can transport a Work Environment between computers, but it is still a filesystem synchronisation mechanism. Avoid simultaneous conflicting edits from multiple computers unless you are certain the synchronisation system has completed.

1.3 The principal data domains

The application can be understood as several connected domains.

Domain	Purpose
Work	Tasks, sub-tasks, Notes and Activities.
Calendar	Scheduled Calendar Events from Testudo or connected external calendars.
Themes	Hierarchical semantic classification of work and entities.
People	Individuals, profiles and professional affiliations.
Groups	Teams, laboratories, committees and other group structures.
Organizations	Institutions and organizational hierarchies.
History	Chronological record of important work-related changes.
Work Environments	Independent portable collections of data, membership and configuration.

Installation

2.1 System requirements

The current public Testudo source targets the modern Apple development platform.

It requires:

- macOS 26 or later;
- an Apple Silicon Mac;
- Xcode 27 or an equivalent Apple developer toolchain;
- Apple Swift 6.4 or later;
- Git for the normal source-based installation workflow.

Older macOS and Xcode releases are not maintained as compatibility targets for the current source tree.

Testudo uses current native SwiftUI functionality, including the modern macOS Liquid Glass interface.

The current distribution model intentionally builds the application locally from source.

A paid Apple Developer Program membership is not required for this local-build installation workflow.

2.2 Quick local installation

The simplest installation workflow is:

```
curl -fsSL https://raw.githubusercontent.com/foivoskar/Testudo/main/install.sh | bash
```

The installer:

1. verifies that the computer is running macOS;
2. verifies that it is an Apple Silicon Mac;
3. checks the Apple developer tools;
4. checks the Swift version;
5. obtains the public Testudo source code;
6. builds the application locally;
7. ad-hoc signs the locally compiled application;
8. creates a local DMG installer;
9. verifies the DMG;
10. opens the finished installer.

The Testudo executable itself is therefore compiled on the user's Mac rather than downloaded as a prebuilt application binary.

Before installing the current source, update macOS and Xcode to the supported versions above. An older SDK or Swift toolchain is not a supported build environment.

2.3 Inspecting the installer first

Users who do not wish to pipe a remote script directly to a shell can download and inspect it first:

```
curl -fsSL \  
  https://raw.githubusercontent.com/foivoskar/Testudo/main/install.sh \  
  -o testudo-install.sh
```

Inspect the file and then run:

```
bash testudo-install.sh
```

2.4 Manual local build

The equivalent manual procedure is:

```
git clone https://github.com/foivoskar/Testudo.git  
cd Testudo  
./Scripts/build-local-dmg.sh
```

After a successful build, the installer is written to the `dist` directory.

For example:

```
dist/Testudo-0.1.4-macOS-arm64.dmg
```

Open the resulting DMG and drag `Testudo.app` into `Applications`.

2.5 Installing Apple developer tools

If the Command Line Tools are missing, macOS can request their installation with:

```
xcode-select --install
```

After the installation completes, repeat the Testudo installation command.

2.6 Updating Testudo

If the one-line installer was used, it can be run again.

For a manual Git checkout:

```
cd ~/Testudo  
git pull --ff-only  
./Scripts/build-local-dmg.sh
```

A new locally built DMG is produced.

Replace the older copy of `Testudo.app` in `/Applications` with the newly built application.

2.7 Why Testudo currently uses local compilation

Current Testudo distribution is source-first.

The project does not depend on distribution of a prebuilt, Developer-ID-signed application. This allows users to obtain the public source and compile the executable directly on their own Mac.

Git tags may be used to identify historical source versions without requiring downloadable application binaries.

First Launch and the Local User

3.1 The local Testudo profile

A Testudo installation has a local application user profile.

The local profile describes the person using this installation and may include professional information such as:

- first, middle and last name;
- preferred name;
- academic title;
- job title;
- professional e-mail addresses;
- telephone numbers;
- office;
- employee identifier;
- professional address;
- website;
- ORCID;
- GitHub;
- LinkedIn;
- ResearcherID;
- Scopus Author ID;
- Google Scholar URL;
- professional fields;
- responsibilities;
- preferred language;
- preferred time zone;
- tags and notes;
- profile image.

This profile belongs to the application installation.

It is conceptually separate from a Person object inside a particular Work Environment.

3.2 Local user versus Environment member

This distinction is important.

The local application user represents the person operating this installation of Testudo.

An Environment membership represents an identity *inside a specific Work Environment*.

The same local user may therefore have different roles in different Environments.

For example:

- Environment A: Administrator;

- Environment B: User;
- Environment C: no membership yet.

3.3 Portable user profile

Testudo supports portable profile files with the extension:

`.testudouser`

A Testudo user profile can be exported and later imported.

This is intended to make restoration of personal profile information easier after a complete local sign-out or when configuring another installation.

What a portable profile does not contain

Environment registrations, Environment passwords and local authentication credentials are intentionally separate from the portable user profile.

Work Environments

4.1 Concept

A **Work Environment** is an independent Testudo workspace.

It contains the work, people, themes, structures, calendar relationships, membership information and history belonging to one logical environment.

Different Environments are isolated from one another.

4.2 The `.testudoenv` package

A Work Environment is stored as a portable package whose filename ends in:

`.testudoenv`

A typical package contains files such as:

```
Example.testudoenv/  
|-- EnvironmentManifest.json  
|-- EnvironmentData.json  
|-- EnvironmentCredentials.json  
'-- EnvironmentIcon.png
```

The Environment icon is optional.

4.3 Environment manifest

The manifest makes a Work Environment self-identifying.

It contains information such as:

- the Environment identifier;
- the Environment name;
- schema information;
- identity-provider information;
- creation and update timestamps.

4.4 Environment storage location

The application stores local knowledge of where each registered Environment package can be found.

macOS persistent file-access bookmarks are used so that Testudo can continue to access a user-selected package where permitted by the operating system.

This local registration is different from the portable Environment data itself.

4.5 Creating or registering an Environment

A user may create a new Work Environment or register an existing `.testudoenv` package.

When moving an Environment between folders or computers, the new Testudo installation may need to be shown the new package location.

4.6 Environment icon

Each Work Environment may have its own icon.

If no custom Environment icon exists, Testudo uses its standard default Environment visual.

Environment Administrators can change or restore the Environment icon through the Environment administration tools.

4.7 Backing up an Environment

Backing up a Work Environment normally means backing up its complete `.testudoenv` package.

Do not back up only one JSON file unless you understand exactly which parts of the Environment you intend to preserve.

Keep independent backups of important Work Environment packages, especially while using early Testudo releases. A synchronised cloud folder is useful for replication, but it should not be your only backup.

Authentication and Roles

5.1 Two authentication layers

Testudo distinguishes between:

1. application-level identity and authentication;
2. Work Environment identity and authentication.

These layers serve different purposes.

5.2 Environment roles

The current Environment role model contains two principal roles:

Administrator Can perform Environment administration operations.

User Standard Environment member.

A person's role belongs to the Environment, not to the application as a whole.

5.3 Environment passwords

Environment users can authenticate using Environment-local accounts.

Users can change their own Environment password.

Environment Administrators can assign a new password to another Environment member without knowing that person's existing password.

5.4 Security questions and password recovery

Testudo supports recovery through security questions.

The current design uses three security questions for account recovery.

Users should choose answers that they can reproduce reliably.

5.5 Stay signed in

A local installation may remember that a user wishes to stay signed in to an Environment.

This remembered local state belongs to the application installation and is not stored as the user's plaintext password inside the portable Environment.

5.6 Log Out and Sign Out

The application distinguishes between leaving an authenticated session and removing broader local application state.

A normal logout should not be confused with complete local sign-out or removal of local profile information.

Before performing a complete sign-out, consider exporting the `.testudouser` profile if you want an easy way to restore profile information later.

The Testudo Interface

6.1 Three-column navigation

Testudo uses a native macOS multi-column interface.

The three conceptual regions are:

1. **main navigation** on the left;
2. **lists, trees or collections** in the middle;
3. **detail view** on the right.

The exact content depends on the currently selected area.

6.2 Main navigation

The current sidebar provides:

- New Entry;
- Today;
- Calendar;
- All Tasks;
- To Do;
- In Progress;
- Completed;
- Archive;
- Timeline;
- Themes;
- Organizations;
- Groups;
- People.

6.3 Sidebar visibility

Testudo uses the native macOS sidebar visibility control.

When the sidebar is visible, the control belongs to the upper-right area of the sidebar. When the sidebar is hidden, macOS relocates the control beside the standard window controls.

On the current macOS interface the control uses the native Liquid Glass appearance.

6.4 Detail navigation

Objects displayed inside detail views are active references wherever navigation is semantically appropriate.

The current navigation system supports direct opening of:

- Tasks;
- Notes;
- Activities;
- Themes;
- People;
- Groups;
- Organizations;
- Calendar Events.

A displayed description or preview that belongs to a particular object can also navigate to the owning object.

For example, the body preview shown for Related Work opens that Work item, while a displayed Theme name opens the Theme itself.

Editors, selection popovers, menus, disclosure controls and similar interactive controls retain their original editing or selection behaviour.

The detail area maintains Back and Forward navigation history so that users can follow chains of references without losing their previous context.

Two small independent circular Liquid Glass buttons are positioned at the upper-right of the right-hand detail pane.

Supported trackpad gestures use the same Back and Forward history.

The navigation history follows the content actually displayed in the right-hand pane rather than only Work-item selection. Tasks, Notes, Activities, Themes, People, Organizations, Groups, Calendar Events and the supported creation views can therefore participate in the same Back and Forward chain.

An explicit selection always takes precedence over a transient creation view, so selecting a concrete object immediately opens that object in the right-hand pane.

6.5 Hierarchical presentation

Hierarchical data is used extensively.

Examples include:

- Tasks and sub-tasks;
- Themes and sub-themes;
- Organizations;
- Groups and their containers;
- People inside Groups or Organizations.

Disclosure controls can be used to expand or collapse nested content.

Hierarchy affects both structural navigation and contextual presentation. In particular, Related Work can be aggregated from descendants into a parent Theme, Organization or Group.

6.6 Middle-column selection

Selection in the middle column is controlled by Testudo's application state rather than depending on temporary macOS List focus.

The selected row fills the complete visible row width and height using Testudo's blue selection colour with subtle rounded corners.

Hierarchical DisclosureGroup rows use the same visual treatment, so Themes and nested Work remain consistent with ordinary List rows.

Work

7.1 New Entry

New Entry is Testudo's global Work creation action.

Selecting New Entry in the sidebar opens the New Work Entry editor directly in the right-hand pane, independently of the previously selected sidebar area.

The keyboard shortcut is:

Command-N

The editor can create a Task, Note or Activity and exposes the relevant Theme, Parent Task, timing, time-zone and relationship controls.

After cancelling a creation form, selecting New Entry again opens a fresh creation form.

7.2 Work item types

The current Work model has three principal item types:

Task An actionable work item with state and optional planning information.

Note Information attached to the Work hierarchy.

Activity A chronological record of something that happened while working.

7.3 Tasks

Tasks can be top-level items or sub-tasks.

A Task may contain:

- sub-tasks;
- Notes;
- Activities.

Sub-tasks may themselves contain further nested Work, allowing complex Work structures without a fixed practical hierarchy depth.

A Task can participate in semantic relationships with People, Groups and Organizations and can belong to one or more Themes.

7.4 Task states

The implemented Task states are:

State	Meaning
To Do	Work exists but is not currently marked as in progress.
In Progress	Work is actively underway.
Completed	The work has been completed.
Closed	The item has been formally closed.

The interface uses a consistent visual status language for the main active states. In particular, To Do, In Progress and Completed have distinct visual treatments.

7.5 Task dates

Depending on the item and workflow, Testudo can maintain timestamps such as:

- creation time;
- update time;
- scheduled time;
- deadline;
- reminder time;
- start time;
- completion time;
- logged time.

These timestamps are stored as absolute instants while preserving relevant source time-zone information.

7.6 Sub-tasks

Tasks can be nested.

This allows larger work items to be decomposed into increasingly specific actions while retaining their relationship to the parent Task.

A sub-task remains a full Task and may have its own:

- state;
- dates;
- content;
- relationships;
- history;
- nested children.

7.7 Notes

Notes are informational Work items.

They can be placed inside the Work hierarchy and used to preserve context that is not itself an actionable Task.

Notes may also participate in date/reminder workflows where supported by the interface.

7.8 Activities

An Activity records something that happened during work.

Examples include:

- a discussion took place;
- a server was upgraded;
- a manuscript section was reviewed;
- a measurement was completed;
- a decision was made;
- an external dependency was resolved.

An Activity is a work-log object, not a scheduled Calendar Event.

This distinction is fundamental:

Activity = something recorded as having happened

Calendar Event = a calendar-based event with a time interval

Relationships

8.1 Why relationships matter

Testudo is not limited to placing Tasks in folders.

Work can be connected semantically to People, Groups and Organizations.

This makes it possible to represent *why* an entity is connected to work.

8.2 Work relationship roles

The current relationship roles are:

Relationship	Typical interpretation
For	The work is being performed for the related entity.
Requested by	The related entity requested the work.
With	The work is being performed with the related entity.
Assigned to	Responsibility for the work is assigned to the related entity.
Related to	A general semantic relationship.

8.3 Relationship inheritance

Work relationships can be inherited by child Work items.

This allows a relationship established at a higher level of the Work hierarchy to remain relevant to nested Work without repeatedly entering the same relationship.

Inheritance is controlled by the **Inherited by children** property of the relationship.

Where precision is required, relationships can also be managed at the specific Work-item level.

8.4 Related Work

People, Themes, Organizations and Groups use a common **Related Work** pane at the top of the right-hand detail area.

When Related Work exists, the detail area is divided into two independently scrollable halves:

- the upper half contains Related Work;
- the lower half contains the normal detail inspector.

When no Related Work exists, the normal detail inspector occupies the full height.

Related Work may contain all three Work types:

- Tasks;
- Notes;

- Activities.

The shared compact row presentation can include:

- Work icon;
- title;
- Work type;
- Task status where applicable;
- Theme;
- deadline where applicable;
- description preview;
- creation date at the right-hand side.

The same typography, spacing and navigation behaviour is used in all four structural areas.

References inside Related Work remain navigable.

Aggregation and inheritance

Related Work aggregation and Work-relationship inheritance are distinct mechanisms.

Relationship inheritance determines whether a child Work item inherits a semantic relationship from a parent Work item.

Related Work aggregation determines which Work items are displayed when viewing a parent Theme, Organization or Group.

Both mechanisms can operate at the same time.

Themes

9.1 Purpose

Themes provide a semantic classification system that is independent of the Work hierarchy and independent of the organizational hierarchy.

A Theme can represent, for example:

- a scientific subject;
- a research programme;
- an infrastructure topic;
- a long-running initiative;
- an administrative domain;
- a technical area.

9.2 Theme hierarchy

Themes can contain sub-themes.

This makes it possible to model broad and narrow areas without forcing the Work hierarchy itself to reproduce the same structure.

9.3 Theme information

The current Theme inspector exposes:

- name;
- parent Theme;
- summary;
- notes;
- related People, Groups and Organizations;
- start date;
- target date;
- tags;
- URL;
- SF Symbol;
- created and updated timestamps.

Older Environment data can contain additional legacy structure metadata. Structure code, status, priority and owner fields remain compatible at the persistence layer but are not exposed by the current Theme interface.

The normal Theme inspector is read-only. A single Edit action in the Theme header opens the complete Theme editor. This replaces the former per-property pencil controls and provides one consistent edit workflow for the whole Theme.

9.4 Themes and Work

Work items can be associated with Themes.

The current data model supports explicit multi-Theme membership, allowing one Task, Note or Activity to belong to more than one semantic domain without duplicating the Work item.

For compatibility with older data, Testudo can still interpret legacy single-Theme membership. Explicit multi-Theme membership is authoritative when present.

The Related Work view of a Theme is hierarchical.

A Theme displays:

- Work belonging directly to that Theme;
- Work belonging to its child Themes;
- Work belonging to all deeper descendant Themes.

Aggregation is recursive.

For example:

```
Theme A
'-- Theme B
    '-- Theme C
```

A Note belonging to Theme C is visible in the Related Work panes of Theme C, Theme B and Theme A.

The Work item's actual Theme membership is not changed by this presentation. Parent Themes aggregate descendant context rather than reassigning the Work item.

9.5 Themes and entities

Themes can also be related directly to People, Groups and Organizations.

This lets Testudo represent knowledge such as:

- which laboratory works on a Theme;
- which person is associated with a research subject;
- which Organization participates in a programme.

People

10.1 People directory

The People area maintains Person entities inside a Work Environment.

A Person may contain professional information and can participate in work relationships, affiliations, groups and organizational structures.

10.2 Names and ordering

The People list is ordered primarily by surname, with first name used as a secondary key where appropriate.

Older records that do not contain a separately stored surname can fall back to their available display name.

10.3 Affiliations

A Person can have multiple affiliations.

This is important for academic and technical work where one individual may simultaneously belong to:

- a university;
- a laboratory;
- a department;
- a collaboration;
- a working group;
- another institution.

10.4 Primary affiliation

One affiliation may be explicitly marked as the primary affiliation.

When no explicit Primary affiliation is selected, Testudo can use an appropriate directly affiliated Organization as the display affiliation.

When available, an Organization's Short Name can be used in compact presentation.

10.5 Related Work

The upper part of a Person detail view uses the common Related Work presentation.

It can display Tasks, Notes and Activities related to the Person.

It also respects Work relationships inherited from parent Tasks when the corresponding relationship is configured to be inherited by children.

Each row uses the same compact typography and navigation behaviour as Related Work elsewhere in Testudo, including the Work item's creation date at the far right.

10.6 Person versus local profile

A Person inside an Environment is not automatically identical to the application's Local User Profile.

A user's compatible local profile information may be used to create or populate a corresponding Person when appropriate, but the two objects belong to different layers of the application.

Groups and Organizations

11.1 Organizations

Organizations represent institutional structures.

Examples include:

- universities;
- institutes;
- departments;
- companies;
- observatories;
- research centres.

Organizations can contain other Organizations, allowing institutional hierarchies to be represented.

11.2 Organization metadata

The current Organization inspector exposes:

- full name;
- short name;
- structural memberships;
- affiliated People;
- start date;
- target date;
- website;
- e-mail;
- phone;
- address;
- city;
- postal code;
- country;
- tags;
- SF Symbol;
- notes;
- created and updated timestamps.

Older Environment data can contain additional legacy structure metadata. Structure code, status, priority and owner remain compatible at the persistence layer but are not exposed by the current Organization or Group interface.

Organizations and Groups use the same whole-object editing pattern as People. Their normal inspectors are read-only, and a single Edit action in the header opens the complete editor, including structural memberships and affiliated People.

11.3 Groups

Groups represent teams or other collective structures.

Examples include:

- laboratories;
- project teams;
- committees;
- working groups;
- technical teams;
- collaborations.

A Group may exist inside an Organization or independently where the data model permits it.

11.4 Membership

Entity memberships connect People, Groups and Organizations.

These memberships are separate from Work relationships.

For example, "Person A belongs to Laboratory B" is structurally different from "Task C is assigned to Person A".

11.5 Related Work

Organizations and Groups use the same Related Work pane as People and Themes.

The lower detail inspectors for Themes, Organizations, Groups and People use a consistent card-based visual language and aligned vertical spacing.

The view can contain Tasks, Notes and Activities.

For container entities, Related Work is aggregated recursively from the complete descendant Entity hierarchy.

An Organization can therefore display:

- Work related directly to the Organization;
- Work related to a child Organization;
- Work related to a descendant Group;
- Work related to a Person contained below it;
- Work inherited through parent-Task relationships where **Inherited by children** is enabled.

A Group likewise aggregates Related Work from its descendant entities where such memberships exist.

A Work item is presented once even when more than one structural path makes it relevant.

This aggregation does not change Entity memberships or Work relationships. It gives parent structures a consolidated view of Work taking place below them.

Calendar

12.1 Calendar Events

Calendar Events represent scheduled events.

A Calendar Event can contain information such as:

- title;
- notes;
- location;
- start time;
- end time;
- all-day state;
- time zones;
- confirmation status;
- recurrence information;
- organizer;
- attendees;
- conference URL;
- external calendar identifiers.

12.2 Calendar Event status

Calendar Events can represent states including:

- Confirmed;
- Tentative;
- Cancelled.

12.3 Testudo calendars

Testudo supports its own local calendar source.

Calendar Events created as local Testudo events can participate directly in Testudo relationships.

12.4 Apple Calendar and iCloud

A Work Environment can connect supported Apple Calendar calendars through macOS EventKit.

Apple Calendar integration requires macOS Calendar permission.

External source data imported from Apple Calendar is treated as read-only source information inside Testudo.

Testudo does not need to modify the original external Apple Calendar event in order to create Testudo-specific relationships.

12.5 iCal subscriptions

Testudo also supports iCal subscription sources.

A subscription can import calendar information from an iCalendar feed.

12.6 Secret iCal addresses

The actual secret iCal subscription URL is not intended to be stored as ordinary portable Environment data.

Testudo stores the sensitive address using the macOS Keychain mechanism and keeps a lookup reference in its calendar configuration.

12.7 External calendars and Environment administration

Connected calendar sources can be managed from the Work Environment administration tools.

Depending on the source, administrators can:

- connect calendars;
- synchronise them;
- review synchronisation status;
- remove an external calendar connection.

Removing a connected external calendar removes its imported Testudo representation and Testudo-specific relationships as appropriate.

It does not imply deletion of the source calendar from the external calendar service.

12.8 Linking Calendar Events to Work

Calendar Events can be linked to Testudo Work items.

This lets a scheduled meeting or appointment be connected to the Task, Note or Activity context to which it relates.

Calendar Events can also be linked to Themes.

Displayed linked Work and Theme references are navigable.

Calendar Events themselves also participate in Testudo's normal detail navigation and Back/Forward history.

Time Zones and Date Handling

13.1 Device time zone

Testudo uses the Mac's automatically updating current time zone as the default display context.

The device zone is obtained from macOS rather than being permanently hard-coded into an Environment.

13.2 IANA time zones

Where a date and time is entered with a source time zone, Testudo uses real IANA time-zone identifiers.

Examples include:

```
Europe/Paris  
America/Argentina/Buenos_Aires  
America/New_York  
Asia/Tokyo
```

13.3 Absolute timestamps

Dates represent absolute instants.

Where relevant, Testudo additionally records the IANA time zone in which that time was originally entered or generated.

13.4 Displaying source and local time

When a timestamp's source zone differs from the current device zone, Testudo can display both contexts.

Conceptually, a time may be shown as:

current-device-time · source-time and source-zone

This is useful when work crosses geographical time zones.

13.5 Changing the selected source zone

When editing a date and changing its source zone, Testudo treats the change as a reinterpretation of the entered wall-clock time.

For example, changing an entry from "10:00 Paris" to "10:00 New York" is treated as a request for 10:00 in the new zone, rather than silently converting the original instant to 04:00 New York.

This makes the time-zone selector behave like an input-zone selector rather than an accidental conversion command.

History and Audit Information

14.1 Work history

Testudo maintains chronological history for important Work changes.

History can include events such as:

- creation;
- editing;
- status changes;
- scheduling;
- starting work;
- completion;
- Activity logging;
- relationship creation;
- relationship removal;
- movement within the Work hierarchy.

14.2 Actor information

Where an authenticated Environment membership is available, history can identify the Environment member responsible for the action.

14.3 History time zones

History timestamps can preserve the zone in which an action occurred.

Where the history describes another date value, Testudo can also retain that value's source zone separately.

Demo Environment

15.1 Purpose

The Work Environments screen provides a way to create a demonstration Environment.

The Demo Environment is intended for learning and exploration without requiring the user to enter a large amount of data manually.

15.2 Demo subject matter

The generated demonstration data represents a fictional academic and scientific working environment.

It can include areas such as:

- Research Software Engineering;
- Scientific Computing and HPC;
- Data Infrastructure;
- AI and Data Science;
- Earth and Environmental Sciences;
- Physics and Astronomy;
- Computational Biology;
- Chemistry and Materials Science;
- Instrumentation and Sensors;
- Open Science and FAIR Data;
- International Collaborations;
- Seminars and Training;
- Research Administration.

15.3 Generated objects

A Demo Environment contains interconnected examples of:

- People;
- Organizations;
- Groups;
- Themes;
- Tasks;
- sub-tasks;
- Notes;
- Activities;
- Calendar Events;
- affiliations;
- Work relationships;

- history.

15.4 Demo dates

Demo dates are generated relative to the date on which the Demo Environment is created.

The generated timeline covers approximately one month before creation and two months after creation.

Operational dates are generally placed on working days and during normal working hours.

15.5 Demo naming

Repeated Demo creation uses the lowest available name.

For example:

```
Demo
Demo-2
Demo-3
```

If a numbered Demo is removed, its number may later become available again.

Data Portability and Storage

16.1 What is portable

The principal portable object is the `.testudoenv` Work Environment package. It contains the Environment data needed to identify and operate that Environment.

16.2 What is local to one installation

Some information intentionally remains local to the Testudo installation.

Examples include:

- local Environment registration;
- filesystem bookmark information;
- local mapping between the application's user and an Environment membership;
- remembered local authentication state;
- some sensitive Keychain-managed credentials.

16.3 Moving a Work Environment

To move an Environment:

1. make sure Testudo has finished writing changes;
2. copy or move the complete `.testudoenv` package;
3. allow filesystem or cloud synchronisation to finish;
4. register or reopen the package from the new location.

16.4 Using cloud-synchronised storage

A Work Environment package can live in a directory managed by a synchronisation provider.

This can make the same Environment package available on more than one computer.

However, users should avoid generating simultaneous divergent edits to the same package.

File synchronisation is not equivalent to historical backup. Accidental deletion or corruption may itself be synchronised. Maintain independent backups for important Environments.

Administration

17.1 Environment Administrator

The Administrator role applies to a particular Work Environment.

Administrative capabilities include Environment-level configuration that ordinary members should not necessarily be able to perform.

17.2 Membership administration

Administrators can manage Environment membership and authentication information according to the current local-account model.

17.3 Password administration

An Administrator can assign a new password to another Environment member without needing to know the old password.

17.4 Environment appearance

Administrators can manage the Work Environment icon, including restoring the default appearance.

17.5 Connected calendars

Environment administration includes management of connected external calendar sources.

This includes supported Apple Calendar and iCal connections.

Security and Privacy

18.1 Local-first privacy model

Testudo does not require a central Testudo cloud service for normal operation.

Environment information remains in the chosen `.testudoenv` package.

18.2 Passwords

Passwords are not intended to be stored as readable plaintext.

The authentication layer uses password hashing and separate recovery mechanisms.

18.3 Sensitive external calendar information

Secret iCal subscription addresses are treated as sensitive information and are stored using macOS Keychain integration rather than being exposed as ordinary Environment data.

18.4 Calendar permission

Apple Calendar integration requires the user to grant the application appropriate macOS Calendar access.

The external Apple Calendar event source remains read-only from Testudo's integration perspective.

18.5 Portable files

A portable Work Environment may contain significant professional or organizational information.

Treat `.testudoenv` packages as potentially sensitive files when copying, sharing or backing them up.

Troubleshooting

19.1 The build fails with an old toolchain

The current Testudo source targets macOS 26 and the contemporary Apple SwiftUI SDK.

Check the selected development tools:

```
xcodebuild -version
swift --version
```

The documented baseline is Xcode 27 with Apple Swift 6.4 or later.

If older versions are installed, update macOS and Xcode before retrying the build.

19.2 SwiftPM build progress

Current Testudo uses SwiftPM's modern default build backend.

During a normal build SwiftPM may display progress such as:

```
[Planning deferred tasks]
```

This text is build progress and is not itself an error.

The current production baseline has been verified to complete normally with the supported toolchain.

19.3 Apple developer tools are missing

Run:

```
xcode-select --install
```

After installation, retry the build.

19.4 The automatic installer refuses to update

The automatic installer intentionally avoids overwriting a source checkout containing local changes.

Inspect the repository:

```
cd ~/Testudo
git status
```

Resolve, commit, stash or deliberately discard your own changes before using automatic update behaviour.

19.5 The Work Environment cannot be found

If a `.testudoenv` package was moved, renamed or relocated by a synchronisation provider, Testudo's stored local path or filesystem bookmark may no longer point to the correct location.

Locate the package and register or reopen it from its current location.

19.6 Apple Calendar does not appear

Check:

1. that Calendar permission has been granted in macOS;
2. that the relevant external calendar has been connected;
3. that the calendar is enabled;
4. that synchronisation has been run successfully.

19.7 iCal synchronisation fails

Possible causes include:

- an invalid or expired subscription address;
- loss of network connectivity;
- a remote server error;
- authentication or access changes at the provider.

Review the calendar's synchronisation status and reconnect the source if necessary.

19.8 Several old Testudo installer volumes appear

Current local-build scripts attempt to detach older mounted Testudo installer images before opening the newly built DMG.

If an old volume remains mounted, eject it manually in Finder or Disk Utility and run the build again.

Recommended Working Practices

20.1 Use Tasks for actionable work

Create a Task when an item represents work that can meaningfully progress through a state.

Use sub-tasks when decomposition adds useful structure.

20.2 Use Notes for context

Use Notes for durable contextual information that should live beside the work but does not itself require Task state.

20.3 Use Activities for the work log

Use Activities to record what actually happened.

A useful Activity should usually answer at least one of these questions:

- What happened?
- When did it happen?
- What changed?
- What decision was made?
- What result was obtained?

20.4 Use Calendar Events for scheduled time

Use Calendar Events when the object belongs on a calendar and has a scheduled time interval.

Do not use Calendar Events merely as a substitute for Activity history.

20.5 Use Themes for semantic classification

Themes are most useful for concepts that cut across the Work, People and Organization hierarchies.

A Work item may belong to more than one Theme when several semantic classifications are genuinely useful.

Because a parent Theme's Related Work includes Work from descendant Themes, broad Themes can provide an overview without requiring Work to be duplicated or manually assigned at every ancestor level.

20.6 Model People and Organizations explicitly

When a person, laboratory or institution repeatedly appears in your work, represent it as an entity rather than repeatedly typing its name into free text.

This makes Testudo relationships navigable and reusable.

20.7 Preserve history

For long-running work, recording Activities and maintaining Task state provides a much more useful record than continually rewriting a single Note.

Technical Reference

21.1 Important file extensions

Extension	Purpose
<code>.testudoenv</code>	Portable Testudo Work Environment package.
<code>.testudouser</code>	Portable local Testudo profile information.
<code>.dmg</code>	Locally generated macOS installer disk image.

21.2 Source tree

Important project locations include:

```
Sources/Testudo/  
|-- Auth/  
|-- Demo/  
|-- Models/  
|-- Resources/  
|-- Store/  
|-- Time/  
|-- Views/  
'-- TestudoApp.swift
```

21.3 Build scripts

The principal build scripts are:

```
Scripts/build-app.sh  
Scripts/build-release.sh  
Scripts/build-local-dmg.sh  
Scripts/version.sh
```

The standard local user-facing build is:

```
./Scripts/build-local-dmg.sh
```

21.4 Version information

The default application version and build number are maintained in:

```
Scripts/version.sh
```

This file acts as the build scripts' central version source.

21.5 Application build

To create the application bundle directly:

```
./Scripts/build-app.sh
```

The resulting bundle is created under:

```
dist/Testudo.app
```

The current build script uses SwiftPM's modern default build backend and does not force the deprecated legacy `-build-system native` option.

The current production baseline is maintained as a warning-free build with the supported Xcode 27 and Apple Swift 6.4 toolchain.

21.6 Local DMG build

To create the installer disk image:

```
./Scripts/build-release.sh
```

or use the complete user-facing workflow:

```
./Scripts/build-local-dmg.sh
```

Glossary

Activity A chronological Work item recording something that happened.

Calendar Event A scheduled event represented in the Calendar domain.

Entity A Person, Group or Organization.

Environment Administrator A member with administrative privileges inside one Work Environment.

Environment member An identity belonging to a specific Work Environment.

Group A team, laboratory, committee, collaboration or similar collective entity.

Local User Profile The professional profile belonging to the user of one Testudo installation.

Note An informational Work item.

Organization An institutional entity that may contain other Organizations or Groups.

Person An individual represented within a Work Environment.

Related Work The common upper detail-pane view used for People, Themes, Groups and Organizations. It presents relevant Tasks, Notes and Activities and can aggregate Work from descendant structures.

Task An actionable Work item with Task status.

Theme A hierarchical semantic classification independent of the Work and organizational hierarchies.

Work Environment An independent portable Testudo workspace represented by a `.testudoenv` package.

Work relationship A semantic connection between a Work item and a Person, Group or Organization.

Command Reference

A.1 Install Testudo

```
curl -fsSL \  
https://raw.githubusercontent.com/foivoskar/Testudo/main/install.sh \  
| bash
```

A.2 Clone manually

```
git clone https://github.com/foivoskar/Testudo.git  
cd Testudo
```

A.3 Update source

```
git pull --ff-only
```

A.4 Build local installer

```
./Scripts/build-local-dmg.sh
```

A.5 Build application only

```
./Scripts/build-app.sh
```

A.6 Check Xcode and Swift

```
xcodebuild --version  
swift --version
```

The documented baseline is Xcode 27 and Apple Swift 6.4 or later.

A.7 Request Apple Command Line Tools

```
xcode-select --install
```

Manual Source and Compilation

The source files for this manual are stored under:

```
Documentation/TestudoManual/
```

The two maintained manual source files are:

```
Testudo_manual.tex  
testudomanual.cls
```

The cover uses:

```
Sources/Testudo/Resources/testudo_icon_blue.png
```

When compiled from the manual directory, the expected PDF is:

```
Testudo_manual.pdf
```

The project-level build procedure places the final copy at the root of the Testudo repository:

```
Testudo_manual.pdf
```

Early Release Notice

Testudo is currently an early public release.

The application, user interface and data model may continue to evolve.

Before upgrading between early versions:

1. ensure important Work Environments are backed up;
2. allow cloud-synchronised copies to finish synchronising;
3. keep an independent copy of critical `.testudoenv` packages;
4. review version-specific changes where relevant.

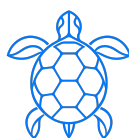
License

Testudo is distributed under the MIT License.

See the repository's [LICENSE](#) file for the complete license text.

Author

Foivos Karakostas



Testudo User Manual
Manual version 1.2
October 2026